

AlphaPi One

完整技术参考手册

版本 v3

2026-08-28

涵盖原板定制固件、通用 MicroPython 固件、小智 AI 语音助手固件三种状态

基于实际逆向研究和测试整理

版本说明

本手册涵盖 AlphaPi One 开发板的三种固件状态：

- 原板（教育系统定制固件）：出厂状态，内置完整教育系统
- 通用版（通用 MicroPython 固件）：刷入通用 MicroPython 后的状态
- 小智固件（AI 语音助手）：刷入 xiaozhi-esp32 固件，变为 AI 语音助手

v3 更新内容（2026-08-28）：

- 新增第八章：小智固件深度逆向分析
- 修正硬件参数：SPI 精确引脚号、N32 音频协议、波特率
- 新增 AI 语音技术栈说明
- 更新对比表（三种固件对比）

一、硬件规格

项目	规格
主控	ESP32-S3, 240MHz 双核, 40MHz 晶振
屏幕	ST7789（兼容 ST7735 命令集），128×160 物理分辨率，SPI 接口
音频协处理器	N32 芯片，通过 UART1 通信
按键	A键、B键、L键、R键、音量±、静音
WiFi	2.4GHz 802.11 b/g/n（仅 2.4G）
蓝牙	BLE（低功耗蓝牙），不支持 A2DP 蓝牙音频
Flash	8MB XMC（OPI 模式）
PSRAM	有（SPIRAM）
USB 桥接	HETAO/N32 芯片，VID:PID 2F4E:0102

屏幕引脚映射（精确值，从小智固件逆向提取）

信号	GPIO
MOSI	**35**
SCK	**37**
MISO	**38**
CS	**33**
DC	**36**
RST	**10**
BL（背光）	**11**

原板通过 `hal.gpio\_map` 获取引脚名（tft_sck/tft_mosi 等），实际数值同上。

音频协处理器引脚

信号	GPIO
TX (ESP32→N32)	GPIO3
RX (N32→ESP32)	GPIO0
波特率	**1,000,000 bps (1Mbps)** (小智固件) / 460929 (原板)

注意：原板固件使用 460929 波特率，小智固件使用 1Mbps。N32 芯片可能支持自适应波特率。

二、原板（教育系统定制固件）

2.1 固件信息

- MicroPython 版本：e388c9f23-dirty on 2023-07-14
- 系统标识：AlphaPi One with ESP32S3
- HT protocol 版本：1215 (旧板) / 231012 (新板)
- 出厂测试标识：testok 文件，内容 `SW\_VER: 23111501 / HW\_NAME: ONE | Nov 15`

2.2 文件系统

原板文件系统包含大量预置文件：

核心程序

文件	大小	说明
boot.py	351B	启动入口，检测 testok 和按键组合
main.py	394B (出厂) / 可自定义	分配 40KB static_buf，调用 ht_main.Start()
ht_main.py	111B (出厂) / 966B (新板)	硬件初始化，屏幕初始化，循环检测硬件
variable.py	62B	变量定义
music.py	31B	音乐相关
pen.py	31B	笔相关

字库文件

文件	大小	说明
HZK16	267616B	16×16 中文字库
gb2312	83607B	GB2312 编码表
sysfont.mpy	2519B	系统字体模块

驱动模块 (.mpy)

文件	说明
ST7735.mpy	屏幕驱动 (8182B)
protocol.mpy	核心协议模块 (UART通信、音频、屏幕、按键)
controlBoardAlphaPiOne.mpy	控制板模块 (20918B)
basic.mpy	基础模块 (含 DataStruct)
static_buf.mpy	静态缓冲区
machine_helper.mpy	机器辅助模块
max30102.mpy	心率血氧传感器驱动
各类传感器模块	超声波、红外、温湿度、光线、雨滴、触摸等

音频文件 (.dat)

文件	说明
alert.dat	提示音
broken.dat	错误音
drop.dat	掉落音
du.dat	嘟音
electric.dat	电流音
funny.dat	趣味音
msg.dat	消息音
pass.dat	通过音
record_start.dat / record_end.dat	录音开始/结束音
right.dat / wrong.dat	正确/错误音
tech.dat	科技音
happy.dat / sad.dat	开心/悲伤音
heart.dat	心跳音

图片文件 (.htbmp / .png / .bmp)

- 各类 .htbmp 图片 (2568B/1448B)：方向箭头、表情、传感器图标等
- logo.png (45074B)
- logo2.bmp (8694B)

2.3 启动流程

```
boot.py
├─ 检测 testok 文件是否存在
├─ 不存在 → 抛出 CustomException
└─ 存在 → boot_key_detect() 检测按键组合 (factory_reset)

main.py
├─ 分配 40KB static_buf = bytearray(40960)
└─ 调用 ht_main.Start(static_buf)

ht_main.py
├─ p.init_files() 初始化文件
├─ p.check_hardware() 检测硬件
├─ 初始化 SPI 屏幕 (TFT 驱动)
├─ 检测 testok 文件
└─ 循环: 检测连接状态, 显示对应界面
```

2.4 屏幕驱动

原板使用 `ST7735.mpy` 驱动, 通过 `protocol` 模块封装。

关键特性

- 分辨率: 128×160
- 通信方式: SPI, 40MHz
- 旋转: 支持 0/1/2/3 四种旋转 (`tft.rotation(n)`)

显示 API (通过 protocol 模块)

API	说明
<code>`p.showString(s)`</code>	显示字符串
<code>`p.showStringWithXY(s, x, y)`</code>	指定位置显示字符串
<code>`p.textzh(s, x, y, size)`</code>	显示中文 (支持 HZK16 字库)
<code>`p.charzh(ch, x, y, size)`</code>	显示单个中文字符
<code>`p.showStringBase64(s)`</code>	Base64 字符串显示
<code>`p.clearScreen()`</code>	清屏
<code>`p.reset_label()`</code>	重置标签
<code>`p.showBmp()`</code>	显示 BMP 图片

底层 TFT API (直接调用 tft 对象)

重要: 原板 TFT API 参数特殊, 需要使用 `DataStruct` 包装, 不能直接传普通参数。

API	正确用法	错误用法
text	<code>`tft.text(DataStruct(...))`</code>	<code>`tft.text(x, y, s, color)`</code>
char	<code>`tft.char(DataStruct(...))`</code>	<code>`tft.char(x, y, ch, color, size)`</code>
fill	<code>`tft.fill(DataStruct(color))`</code>	<code>`tft.fill(color)`</code>

API	正确用法	错误用法
rect	<code>`tft.rect(DataStruct(...))`</code>	<code>`tft.rect(x, y, w, h, color)`</code>
pixel	<code>`tft.pixel(DataStruct(...))`</code>	<code>`tft.pixel(x, y, color)`</code>

``DataStruct`` 定义在 ``basic`` 模块中，用于封装参数传递给底层 C 函数。

2.5 音频系统

原板音频通过 UART1 与 N32 音频协处理器通信，支持播放 flash 中的音频文件。

音频 API (通过 controlBoardAlphaPiOne 模块)

API	说明
<code>`c.playFlash(DataStruct(filename))`</code>	播放 flash 中的音频文件 (同步)
<code>`c.playFlashAsync()`</code>	异步播放
<code>`c.getPlayingFlashName()`</code>	获取当前播放的文件名
<code>`c.set_volume(DataStruct(volume))`</code>	设置音量 (0-100)
<code>`c.read_volume()`</code>	读取当前音量
<code>`c.stopPlay()`</code>	停止播放
<code>`c.play_record_loop()`</code>	播放录音循环

支持的音频格式

- ``dat`` 格式 (自定义音频格式, IMA ADPCM 编码)
- ``wav`` 格式 (16kHz 16-bit mono PCM)
- ``ima`` 格式 (IMA ADPCM)

2.6 按键系统

按键	读取方式	键值
A键	<code>`c.getStatus(DataStruct('0'))`</code>	按下返回 False/0
B键	<code>`c.getStatus(DataStruct('1'))`</code>	按下返回 False/0
L键	<code>`p.uart_read(0,1)[3] & 0x02`</code>	按下为非零
R键	<code>`p.uart_read(0,1)[3] & 0x01`</code>	按下为非零

``keyP`` 数组: ``[128, 64, 32, 16, 8, 4, 2, 1]``, 用于位操作读取按键状态。

2.7 WiFi

原板支持 WiFi, 通过 ``network`` 模块操作。

```
import network
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect('SSID', 'password')
wlan.isconnected() # 检查连接状态
wlan.ifconfig()    # 获取 IP 配置
```

2.8 字库与中文显示

原板内置完整的中文字库支持：

- HZK16: 16×16 点阵中文字库，覆盖 GB2312 字符集
- gb2312: GB2312 编码表，用于 Unicode→GB2312 转换
- sysfont.mpy: 系统字体模块，包含 ASCII 字模

中文显示通过 `p.textzh()` 或 `p.charzh()` 实现，自动从 HZK16 字库中查找字模。

2.9 协处理器协议（底层）

原板 N32 音频协处理器使用自定义二进制协议：

命令	值	说明
写寄存器	0x90	格式: 0x90 [addr] [len] [data...] [checksum]
读寄存器	0x80	格式: 0x80 [addr] [len], 响应: 0x81 [addr] [len] [data...] [checksum]
写响应	0x91	写操作确认
读响应	0x81	读操作返回数据

寄存器地址

地址	名称	说明
0x00	KEY	按键状态
0x10	AUDIO_CTRL	录音控制 (1=开, 0=关)
0x11	AUDIO_READ	读取麦克风 PCM 数据
0x14	VOLUME	音量 (0-100)
0x15	AUDIO_STREAM	音频流写入 (播放 PCM)

音频流播放

- 每块数据 200 字节
- 格式: 16kHz 16-bit signed little-endian mono PCM
- 播放时需暂停录音（半双工）

三、通用版（通用 MicroPython 固件）

3.1 固件信息

- 通用 MicroPython 固件：刷入后覆盖原板定制系统
- 文件系统：空的，需要自行上传文件
- 失去所有原板专有功能：protocol、ST7735、controlBoardAlphaPiOne、字库等

3.2 可用模块

通用版保留 MicroPython 标准库模块：

模块	说明
machine	硬件控制（Pin, SPI, UART, PWM, ADC, RTC, reset）
network	WiFi 控制（WLAN, STA_IF, AP_IF）
socket / usocket	网络套接字
ssl	SSL/TLS 支持（注意：模块名是 `ssl`，不是 `ussl`）
urequests	HTTP 请求库（支持 HTTPS）
ntptime	NTP 时间同步
gc	垃圾回收
os / uos	文件系统操作
time	时间相关
math	数学函数
random	随机数
ubinascii	Base64/十六进制编解码
ujson	JSON 编解码
bluetooth	BLE 蓝牙（仅 GAP/GATT，不支持 A2DP）
struct	二进制数据打包/解包
esp	ESP32 特定功能

3.3 不可用模块（原板专有）

以下模块在通用版中不存在，需要自行实现：

模块	原板功能	通用版替代方案
protocol	核心协议（UART/音频/屏幕/按键）	自行实现 UART1 驱动
ST7735.mpy	屏幕驱动	自行实现 SPI 屏幕驱动
controlBoardAlphaPiOne	控制板（音频/按键）	自行实现
basic（DataStruct）	数据结构封装	不需要（直接传参）
sysfont.mpy	系统字体	自行实现字模或上传字库

模块	原板功能	通用版替代方案
HZK16	中文字库	自行上传字库文件
gb2312	GB2312 编码表	自行上传
static_buf	静态缓冲区	自行分配 bytearray

3.4 屏幕驱动（需自行实现）

通用版没有 ST7735.mpy 驱动，需要自行实现：

```
from machine import Pin, SPI
import st7735 # 需要自行编写或上传驱动

spi = SPI(2, baudrate=40000000, polarity=0, phase=0,
          sck=Pin(37), mosi=Pin(35), miso=Pin(38))
tft = st7735.TFT(spi, dc=Pin(36), rst=Pin(10), cs=Pin(33))
tft.rotation(1)
tft.fill(0x0000)
tft.text('Hello', 10, 10, 0xFFFF)
```

注意：通用版的屏幕驱动可以直接传普通参数，不需要 DataStruct 包装。

3.5 音频驱动（需自行实现）

通用版需要自行实现 UART1 音频驱动，通过 N32 协处理器协议播放 PCM。

硬件配置

- UART1, TX=GPIO3, RX=GPIO0
- 波特率：460929（原板兼容）或 1Mbps（小智固件）
- 协处理器协议与原板相同（0x90 写寄存器、0x80 读寄存器）

关键代码

```

from machine import UART, Pin
import time

uart = UART(1, baudrate=460929, tx=3, rx=0, bits=8, parity=None, stop=1, timeout=50)

def checksum(data):
    s = 0
    for b in data:
        s = (s + b) & 0xFF
    return s

def write_register(addr, value):
    frame = bytearray([0x90, addr, 1, value])
    frame.append(checksum(frame))
    uart.write(frame)
    # 读取 0x91 响应...

def set_volume(vol):
    write_register(0x14, vol) # 0x14 = 音量寄存器

def write_pcm(pcm_bytes):
    """写入16kHz 16-bit mono PCM数据播放"""
    # 暂停录音
    write_register(0x10, 0)
    # 分块写入, 每块200字节
    offset = 0
    while offset < len(pcm_bytes):
        chunk = pcm_bytes[offset:offset+200]
        frame = bytearray([0x90, 0x15, len(chunk)]) # 0x15 = 音频流
        frame.extend(chunk)
        frame.append(checksum(frame))
        uart.write(frame)
        # 读取响应...
        offset += 200
    # 恢复录音 (可选)

```

音频格式限制

- 仅支持 16kHz 16-bit signed little-endian mono PCM
- 不支持 MP3/AAC/WMA 等压缩格式 (通用 MicroPython 没有解码器)
- 需要自行将 MP3 转换为 PCM (可在电脑端用 ffmpeg 预处理)

3.6 按键驱动 (需自行实现)

通用版需要自行实现按键读取:

```

# 方式1: 通过 UART1 协处理器读取按键
def read_key():
    uart.write(bytearray([0x80, 0x00, 1])) # 读寄存器 0x00 (按键)
    resp = uart.read(5) # 0x81 0x00 0x01 [value] [checksum]
    return resp[3] if resp else None

# A键: value & 0x04
# B键: value & 0x08 (需确认)

```

3.7 WiFi 与网络

连接 WiFi

```
import network
wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect('SSID', 'password')
while not wlan.isconnected():
    time.sleep(0.3)
print(wlan.ifconfig())
```

重要：DNS 设置

通用版连接 WiFi 后，DHCP 可能不会正确设置 DNS 服务器，导致域名解析失败（错误 -202）。

必须手动设置 DNS：

```
cfg = wlan.ifconfig()
# (IP, 子网掩码, 网关, DNS)
wlan.ifconfig((cfg[0], cfg[1], cfg[2], '114.114.114.114'))
```

推荐 DNS：

- 国内：`114.114.114.114`、`223.5.5.5`（阿里）
- 国际：`8.8.8.8`（Google，国内可能不稳定）

WiFi 稳定性

- 每次重新测试前建议硬件复位（重新拔插板子）
- 连接后建议等待 1-2 秒再进行网络操作
- 如果连接不稳定，可在连接前先 `wlan.disconnect()`
- `Wifi Internal State Error` 错误：需要先 `active(False)` 再 `active(True)` 重置

3.8 HTTPS 支持

通用版支持 HTTPS：

- SSL 模块名：`ssl`（不是 `ussl`）
- HTTP 库：`urequests`（支持 HTTPS）

```
import urequests
r = urequests.get('https://www.baidu.com')
print(r.status_code)
print(r.content[:100])
r.close()
```

国内可访问的 HTTPS 站点

- [OK] 百度（www.baidu.com）
- [OK] 腾讯（www.qq.com）— 有时不稳定

- [X] Google (8.8.8.8 等) — 国内被墙
- [X] 国外 CDN (jsdelivr 等) — 有时超时

3.9 网络收音机限制

通用版做网络收音机的核心限制：

限制	说明	解决方案
无 MP3 解码器	大多数网络电台是 MP3/AAC 格式	换用 Arduino + ESP32-audioI2S
仅支持 PCM	只能播放 16kHz 16-bit mono PCM	电脑端预处理转换
无流式解码	urequests 会将整个响应加载到内存	大文件会内存不足

推荐方案：换用 Arduino IDE + ESP32-audioI2S 库，内置 Helix MP3/AAC 解码器，支持网络电台流实时解码播放。

四、小智固件 (AI 语音助手) 深度逆向分析

本章基于 `AlphaPi-One-xiaozhi-splash-20260828.bin` 固件的完整逆向分析。

4.1 固件概览

项目	内容
固件名	AlphaPi-One-xiaozhi-splash-20260828
大小	6.54 MB (合并固件)
开发框架	ESP-IDF (C++)，非 MicroPython
协议版本	2024-11-05 (JSONRPC)
OTA 服务器	`https://api.tenclass.net/xiaozhi/ota/`
设计者	@2333
项目名	alphapi-one

4.2 分区布局

分区	偏移	大小	用途
nvs	0x9000	16KB	配置存储 (WiFi、音量、OTA URL)
otadata	0xd000	8KB	OTA 切换数据
phy_init	0xf000	4KB	RF 校准
ota_0	0x20000	3008KB	主程序 (当前运行)
ota_1	0x310000	3008KB	OTA 备用分区
assets	0x600000	2048KB	资源包 (字体、模型、图片)

4.3 显示驱动

项目	参数
驱动芯片	**ST7789** (兼容 One 板子屏幕)
分辨率	320×240 (LVGL 帧缓冲, 实际通过 offset 显示 128×160)
SPI 主机	SPI2_HOST + DMA 自动分配
图形库	LVGL 9
颜色格式	RGB565 (COLMOD=0x55)
背光	GPIO 11 (PWM 可调)

SPI 引脚 (精确值) :

功能	GPIO
MOSI	35
SCK	37
MISO	38
CS	33
DC	36
RST	10
BL	11

初始化流程:

```
spi_bus_initialize(SPI2_HOST, bus_config, SPI_DMA_CH_AUTO)
→ esp_lcd_new_panel_io_spi(SPI2_HOST, io_config, &panel_io)
→ esp_lcd_new_panel_st7789(panel_io, panel_config, &panel)
→ esp_lcd_panel_reset(panel)
→ esp_lcd_panel_init(panel)
→ esp_lcd_panel_invert_color(panel, false)
→ esp_lcd_panel_swap_xy(panel, DISPLAY_SWAP_XY)
→ esp_lcd_panel_mirror(panel, DISPLAY_MIRROR_X, DISPLAY_MIRROR_Y)
→ esp_lcd_panel_disp_on_off(panel, true)
```

4.4 音频系统 (N32 芯片深度协议)

这是最有价值的发现。音频不是 I2S, 而是 UART 外接 N32 音频编解码芯片。

物理层

项目	参数
芯片	N32 (UART 音频编解码器)
接口	UART1
波特率	**1,000,000 bps (1Mbps)**
缓冲区	4096 RX + 4096 TX

项目	参数
采样率	16KHz (语音识别, 主) / 32KHz / 48KHz
麦克风	双麦 (2MIC, AFE 支持)

N32 通信协议

1. 状态查询

发送查询命令 → 返回 status(1B) + 6状态位
N32 status=0x%02x bits=%c%c%c%c%c%c official=%s

有超时检测: `N32 status poll timeout or invalid frame`

2. 时隙 (Slot) 系统

时隙	值	用途
slot0	0x04	麦克风通道 0
slot1	0x08	麦克风通道 1
slot2	0x0c	播放输出
slot3	0x02	控制/状态
slot4	0x01	保留
slot5	0x03	保留

3. 麦克风数据帧

- 帧头 + 时隙 + PCM 数据 + 校验和(1B)
- 校验失败日志: `mic checksum mismatch got=%02x expected=%02x`

4. 写命令 ACK

发送: 地址(1B) + 数据(3B)
等待: ACK 返回
audio ACK failed addr=0x%02x got=%d [%02x %02x %02x]

5. 音量控制

output volume=%d%% (N32 level=%u)

0-100% 映射到 N32 内部 level 寄存器。

音频处理链

麦克风(N32) → UART(1Mbps) → ESP AFE前端 → WakeNet9唤醒词 → Opus编码 → WebSocket/MQTT
↓
服务器TTS → Opus解码 → 重采样 → N32(UART) → 扬声器

4.5 AI 语音技术栈（乐鑫 ESP-SR 框架）

深度学习推理库 (dl_lib)

- 卷积神经网络: atrous_conv1d（空洞卷积）、depthwise_conv1d（深度可分离卷积）
- 注意力机制: dl_layer_attention
- 量化支持: int8/int16 量化推理
- 矩阵运算: dl_lib_matrixq（量化矩阵）

语音算法模型

模型	文件	用途
WakeNet9	wakenet9_quantized.c / wakenet9l_quantized.c	唤醒词检测（9代, 量化版）
MultiNet5	multinet5_quantized8.c	多语言语音命令识别
NSNet2	nsnet2_quantized.c	神经网络降噪
VADNet1	vadnet1_quantized.c	语音活动检测
HufZip	hufzip_model.c	霍夫曼压缩模型

音频前端处理 (AFE)

算法	说明
AEC	回声消除（Acoustic Echo Cancellation）
AGC	自动增益控制（Automatic Gain Control）
NS	降噪（Noise Suppression），含 WebRTC NS
BSS	盲源分离（Blind Source Separation）
SE	语音增强（Speech Enhancement）
VAD	语音活动检测（Voice Activity Detection）

AFE 配置：FD AFE（全双工）模式，NS=off，支持 AEC（需要播放参考通道）。

音频编解码

- Opus 编码器/解码器
- 自动比特率（OPUS_BITRATE_AUTO）
- Ogg 容器支持（OpusHead / OpusTags）
- 帧时长：2ms / 5ms / 60ms

4.6 按键系统

按键	功能
B 键短按	播放/暂停
B 键长按	`start listening` 开始语音监听

按键	功能
B 键抬起	结束说话
音量+	volume_up (delta 检测)
音量-	volume_down
静音	volume_mute / volume_off

4.7 启动流程（完整）

```
app_main()
├─ 读取 UUID / SKU / MAC 地址
├─ AlphaPiOneBoard 构造函数
│   └─ TRACE display gpio → 配置背光 GPIO(11)
│   └─ TRACE display spi init → SPI2 初始化
│   └─ TRACE display panel io → SPI IO 配置
│   └─ TRACE display panel create → ST7789 创建
│   └─ TRACE display panel reset
│   └─ TRACE display panel init
│   └─ TRACE display panel configured → swap_xy/mirror
│   └─ display ready
├─ GetAudioCodec → N32 UART 初始化(1Mbps)
├─ Initialize LVGL library + port (512KB PSRAM 图像缓冲)
├─ Initialized FD AFE → 音频前端(16KHz)
│   └─ WakeNet9 唤醒词模型加载
│   └─ MultiNet5 语音识别模型加载
│   └─ NSNet2 降噪模型加载
│   └─ VADNet1 语音检测模型加载
├─ WifiManager Initialize
│   └─ 有保存WiFi → 连接
│   └─ 无WiFi → 启动AP配网(192.168.4.1)
├─ Audio codec started
└─ 连接服务器 (MQTT/WebSocket)
```

4.8 通信协议

项目	参数
协议版本	2024-11-05
传输层	MQTT 或 WebSocket（可配置）
音频通道	UDP + AES 加密（可选）
MCP 工具	get_device_status, upgrade_firmware

消息类型：

- `hello` - 连接握手
- `goodbye` - 断开
- `wake\_word\_detected` - 唤醒词事件
- `listen` - 开始监听
- `error` - 错误通知

4.9 WiFi 配网

- AP 模式 SSID 动态生成
- 网关: `192.168.4.1`
- 内置多语言配网网页 (20+ 语言)
- 支持保存多个 WiFi, 自动重连
- 2.4G only (`WIFI\_BAND\_MODE\_2G\_ONLY`)
- 高级配置: 自定义 OTA URL、记住 BSSID、省电模式

4.10 NVS 配置存储

键名	用途
`ssid`	WiFi 名称
`password`	WiFi 密码
`remember_bssid`	记住 BSSID
`max_tx_power`	WiFi 发射功率
`sleep_mode`	WiFi 省电模式
`ota_url`	自定义 OTA 地址
`language`	语言设置
`board`	板型标识
`uuid` / `mac` / `mac_address`	设备标识
`wake_word_detected`	唤醒词状态

4.11 电源管理

模式	说明
PERFORMANCE	无省电 (NONE), 性能最优
BALANCED	平衡模式 (MIN_MODEM)
LOW_POWER	低功耗 (MAX_MODEM)

- 支持 light sleep
- WiFi 发射功率可配置 (从 NVS 读取)

4.12 OTA 升级

- 服务器: `https://api.tenclass.net/xiaozhi/ota/`
- 双分区: ota_0 (当前) + ota_1 (备用)
- 下载到备用分区 → 校验 → 切换 → 重启
- 支持 assets 资源包单独升级 (带 MD5 校验)
- 支持 MCP 远程触发升级

4.13 资源包 (assets)

文件	大小	用途
font_noto_sans_common_14_1.bin	548KB	Noto Sans 14号字体 (LVGL格式)
srmodels.bin	28KB	语音识别模型
index.json	-	资源索引
9张 BMP	-	UI 图标 (WiFi信号、音量、电池、箭头等)

4.14 LVGL UI 系统

UI 组件

- 按钮: Btn1 / Btn2 / Btn3 / Btn4 / Btn5
- 复选框: Check box
- 聊天气泡: chat_message_label
- 状态栏: status_label
- 通知: notification_label
- GIF 动画支持
- 截图功能 (screenshot.jpg)

表情/情绪状态

`happy, sad, angry, surprised, neutral, sleepy, thinking, speaking, listening, connecting, error, warning`

4.15 TLS 安全证书

内置 7 类证书:

- Amazon Root CA 3/4
- BR Root CA 1 2020
- COMODO ECC/RSA Certification Authority
- GlobalSign Root
- ECC Root CA (多个版本)
- Certainly Root E1
- EV Root CA 1 2020

支持 RSA + ECC 双算法。

4.16 任务/线程架构

任务名	说明
`main_task`	主任务
`audio`	音频处理
`wifi`	WiFi 管理
`websocket`	WebSocket 通信

任务名	说明
`ipcX`	进程间通信

五、三种固件对比表

特性	原板（定制固件）	通用版（MicroPython）	小智固件（AI助手）
开发语言	MicroPython	MicroPython	C++ (ESP-IDF)
屏幕驱动	[OK] ST7735.mpy	[X] 需自行实现	[OK] ST7789 + LVGL9
中文显示	[OK] HZK16 + textzh	[X] 需自行实现	[OK] Noto Sans 字体
音频播放	[OK] .dat/.wav/.ima	[!] 仅16kHz PCM	[OK] Opus + N32
语音识别	[X]	[X]	[OK] WakeNet9 + MultiNet5
AI 对话	[X]	[X]	[OK] 完整 AI 助手
按键读取	[OK] getStatus API	[X] 需自行实现	[OK] B键长按监听
WiFi	[OK] network	[OK] network（需手动DNS）	[OK] 配网网页
HTTPS	[X]	[OK] ssl + urequests	[OK] mbedTLS
OTA 升级	[X]	[X]	[OK] 双分区 OTA
文件系统	[OK] 预置大量资源	[X] 空文件系统	[OK] assets 资源包
教育系统	[OK] 完整教育功能	[X] 无	[X] 无
开发灵活性	[!] 受限于定制 API	[OK] 完全自由	[!] 需编译固件
DataStruct	[OK] 必需	[X] 不需要	N/A
蓝牙	[OK] BLE	[OK] BLE	[OK] BLE
音频波特率	460929	460929（可改）	1Mbps

六、音频协处理器协议详解

三种固件版本都通过 UART1 与 N32 音频协处理器通信，协议基本相同。

6.1 物理层

- UART1, TX=GPIO3, RX=GPIO0
- 波特率：460929（原板/通用版）或 1,000,000（小智固件）
- 数据位：8，停止位：1，无校验

6.2 写寄存器命令

格式：`0x90 [addr] [len] [data...] [checksum]`

- `0x90`：写命令标识

- `addr`: 寄存器地址 (1字节)
- `len`: 数据长度 (1字节)
- `data`: 写入的数据 (len 字节)
- `checksum`: 校验和 = $(0x90 + \text{addr} + \text{len} + \text{data...}) \& 0xFF$

响应: `0x91 [addr] [checksum]` (3字节)

6.3 读寄存器命令

格式: `0x80 [addr] [len]`

- `0x80`: 读命令标识
- `addr`: 寄存器地址 (1字节)
- `len`: 读取长度 (1字节)

响应: `0x81 [addr] [len] [data...] [checksum]` (len+4 字节)

6.4 寄存器列表

地址	名称	读写	说明
0x00	KEY	读	按键状态, A键=bit2(0x04)
0x10	AUDIO_CTRL	写	录音控制: 1=开启录音, 0=关闭录音
0x11	AUDIO_READ	读	读取麦克风 PCM 数据 (16kHz 16-bit)
0x14	VOLUME	写	音量设置: 0-100
0x15	AUDIO_STREAM	写	音频流写入 (播放), 每块200字节

6.5 小智固件扩展: 时隙系统

小智固件使用更复杂的时隙协议:

时隙	值	用途
slot0	0x04	麦克风通道 0
slot1	0x08	麦克风通道 1
slot2	0x0c	播放输出
slot3	0x02	控制/状态
slot4	0x01	保留
slot5	0x03	保留

6.6 音频播放流程

1. 设置音量: 写寄存器 0x14
2. 关闭录音: 写寄存器 0x10 = 0 (半双工, 播放时不能录音)

3. 写入 PCM 数据：写寄存器 0x15，每块 200 字节
 - PCM 格式：16kHz, 16-bit, signed, little-endian, mono
 - 200 字节 = 100 样本 = 6.25ms 音频
4. 等待每块的 0x91 响应
5. 播放完毕：可选择恢复录音（写 0x10 = 1）

6.7 校验和计算

```
def checksum(data):
    s = 0
    for b in data:
        s = (s + b) & 0xFF
    return s
```

七、常见问题与解决方案

Q1：通用版连接 WiFi 后无法访问外网（错误 -202）

原因：DHCP 没有正确设置 DNS 服务器。

解决：连接后手动设置 DNS：

```
cfg = wlan.ifconfig()
wlan.ifconfig((cfg[0], cfg[1], cfg[2], '114.114.114.114'))
```

Q2：通用版网络操作经常超时

原因：WiFi 连接不稳定，或 DNS 设置未生效。

解决：

1. 每次测试前硬件复位（重新拔插）
2. 连接 WiFi 前先 `wlan.disconnect()`
3. 连接后等待 1-2 秒
4. 设置 DNS 后再等待 1 秒
5. `Wifi Internal State Error`：先 `active(False)` 再 `active(True)`

Q3：原板 tft.text() 报错 "function takes N positional arguments"

原因：原板 TFT API 需要 DataStruct 包装参数，不能直接传普通参数。

解决：使用 protocol 模块的高层 API（showString、textzh 等），或使用 DataStruct 包装参数。

Q4：通用版无法播放 MP3 网络电台

原因：通用 MicroPython 没有 MP3 解码器。

解决：

1. 换用 Arduino + ESP32-audioI2S 库 (推荐)
2. 电脑端将 MP3 转换为 16kHz PCM 后推送
3. 使用云服务器做中转解码

Q5：原板刷通用版后如何恢复原板系统

解决：需要原板固件备份。建议刷机前先备份整个 flash。如果没有备份，需要联系厂商获取原厂固件。

Q6：通用版屏幕不亮或白屏

原因：屏幕驱动未正确初始化，或 SPI 引脚配置错误。

解决：

1. 确认 SPI 引脚映射 (MOSI=35, SCK=37, MISO=38, CS=33, DC=36, RST=10)
2. 确认屏幕初始化序列 (ST7735/ST7789)
3. 确认旋转方向 (rotation 0-3)

Q7：通用版音频有杂音或播放速度不对

原因：PCM 格式不匹配 (采样率、位深、声道数)。

解决：确保 PCM 数据为 16kHz、16-bit、signed、little-endian、mono。

- 用 ffmpeg 转换：`ffmpeg -i input.mp3 -ar 16000 -ac 1 -sample\_fmt s16 output.wav`

Q8：小智固件刷入后屏幕白屏

原因：小智固件使用 ST7789 驱动 + 320×240 帧缓冲，One 板子物理屏幕是 128×160。

解决：

1. 确认屏幕是否为 ST7789 兼容 (大部分是)
2. 检查 SPI 引脚是否正确
3. 可能需要修改 offset 偏移来适配 128×160 分辨率

Q9：N32 音频芯片波特率是多少？

答案：

- 原板固件：460929 bps
- 小智固件：1,000,000 bps (1Mbps)
- N32 芯片可能支持自适应波特率，两种都能工作

八、新板与旧板差异

项目	旧板	新板
HT protocol	1215	231012
main.py (出厂)	394B	7939B (可能已被刷写)
ht_main.py	111B	966B

项目	旧板	新板
HZK16	267616B	267616B
ST7735.mpy	8182B	8182B
testok	存在	存在
硬件	ESP32-S3	ESP32-S3

新板出厂时 main.py 可能已被刷写为自定义程序（如音乐播放器），ht_main.py 也包含更完整的硬件检测逻辑。

九、参考资料

- 小智固件 OTA：`https://api.tenclass.net/xiaozhi/ota/`
- 自定义唤醒词文档：飞书 wiki
- 乐鑫 ESP-SR：语音识别框架（WakeNet/MultiNet/AFE）
- LVGL 9：嵌入式图形库
- ESP-IDF：乐鑫物联网开发框架

本手册基于 AlphaPi One 开发板的实际逆向研究和测试整理，涵盖原板定制固件、通用 MicroPython 固件和小智 AI 语音助手固件三种状态。

v3 更新于 2026-08-28，新增小智固件深度逆向分析章节。

附录 C：核桃编程显示扩展板驱动

基于实际逆向研究和测试整理 | 2026 - 08 - 28

C.1 硬件信息

核桃编程显示扩展板是一块通过金手指插槽与 AlphaPi One 连接的外部显示模块。

硬件规格：

LCD 型号	LX - 12864L - 1（深圳市力先同创电子）
尺寸	2.0 寸
分辨率	128 × 64 点阵
LCD 控制器	ST7565R（原生 SPI 接口）
桥接 MCU	N32G031F8S7（国民技术，I2C 转 SPI）
通信方式	I2C
I2C 地址	0x09
I2C 引脚	SCL = GPIO5，SDA = GPIO4
字符规格	16 列 × 4 行（8×16 字体）

连接方式：将 AlphaPi One 插入显示扩展板的金手指插槽，扩展板通过板上的 N32G031 MCU 将 I2C 信号转换为 SPI 信号驱动 ST7565R LCD。

C.2 协议说明

经过逆向研究，N32G031 的 I2C 协议具有以下特点：

- 哑字节机制**
每次 I2C 发送的第一个字节会被丢弃，因此所有命令和数据前都需要加一个哑字节 0x00。
示例：发送 "HELLO" 实际需要发送 b'\x00HELLO'
- 清屏**
多次发送 64 字节的 0x00 可覆盖所有像素实现清屏。建议连续发送 3 - 4 次以确保稳定。
- 显示文字**
构建 64 字节缓冲区（4行 × 16列），一次性发送利用自动换行显示完整 4 行内容。普通 ASCII（0x20 - 0x7E）可直接显示。
- 显示图标**
命令格式：0x01 + 参数。参数 0x01 显示电池图标，其他参数显示不同图标。
- 屏幕方向**
命令格式：0x02 + 参数。0x01 = 正面显示，0x02 = 倒立显示。
- 自动换行**
发送满 16 个字符后自动换到下一行。一次性发送 64 字节可填满 4 行。

C . 3 完整驱动代码

以下是经过验证的 `MicroPython` 驱动库，保存为 `walnut_lcd.py`：

```
import machine, time

class WalnutLCD:
    def __init__(self, scl=5, sda=4, addr=0x09):
        self.i2c = machine.SoftI2C(
            scl=machine.Pin(scl),
            sda=machine.Pin(sda),
            freq=50000
        )
        self.addr = addr
        self.width = 16
        self.height = 4
        self.buf = bytearray(64)
        self.clear_buf()

    def _send(self, data):
        try:
            self.i2c.writeto(self.addr, b'\x00' + data)
            time.sleep(0.03)
            return True
        except:
            return False

    def clear_buf(self):
        for i in range(64):
            self.buf[i] = 0x20

    def clear(self):
        for _ in range(4):
            self._send(b'\x00' * 64)
            time.sleep(0.05)
        self.clear_buf()

    def set_line(self, row, text):
        if isinstance(text, str):
            text = text.encode()
        text = text[:16]
        start = row * 16
        for i in range(16):
            if i < len(text):
                self.buf[start + i] = text[i]
            else:
                self.buf[start + i] = 0x20

    def set_char(self, row, col, ch):
        if isinstance(ch, str):
            ch = ch.encode()[0]
        if 0 <= row < self.height and 0 <= col < self.width:
            self.buf[row * 16 + col] = ch

    def flush(self):
        self._send(bytes(self.buf))

    def show(self, lines):
        self.clear()
        time.sleep(0.1)
        for i, line in enumerate(lines[:4]):
            self.set_line(i, line)
```

```
self.flush()

def icon(self, icon_id):
    self._send(b'\x01' + bytes([icon_id]))

def flip(self, invert=False):
    if invert:
        self._send(b'\x02\x02')
    else:
        self._send(b'\x02\x01')
```

C . 4 使用示例

基本显示：

```
from walnut_lcd import WalnutLCD
lcd = WalnutLCD()
```

```
# .....4....
lcd.show([
    'Walnut LCD 16x4',
    'Driver v1.0 OK',
    'Temp:26C Hum:55%',
    'IP:192.168.1.100',
])
```

逐行更新：

```
lcd.clear()
lcd.set_line(0, 'Line 0 Updated')
lcd.set_line(1, 'Line 1 Updated')
lcd.set_line(2, 'Line 2 Updated')
lcd.set_line(3, 'Line 3 Updated')
lcd.flush()
```

显示图标和翻转：

```
lcd.icon(0x01) # .....
lcd.flip(True) # ....
lcd.flip(False) # ....
```

注意事项：

- 1 . I 2 C 频率建议使用 50kHz，过高可能导致通信不稳定
- 2 . 每次发送后需要短暂延时（0.03秒）确保 N32G031 处理完成
- 3 . 清屏建议连续发送 4 次 64 字节 0x00 以确保稳定
- 4 . 一次性发送 64 字节缓冲区是最稳定的多行显示方式
- 5 . 该扩展板仅支持 ASCII 字符显示，不支持中文